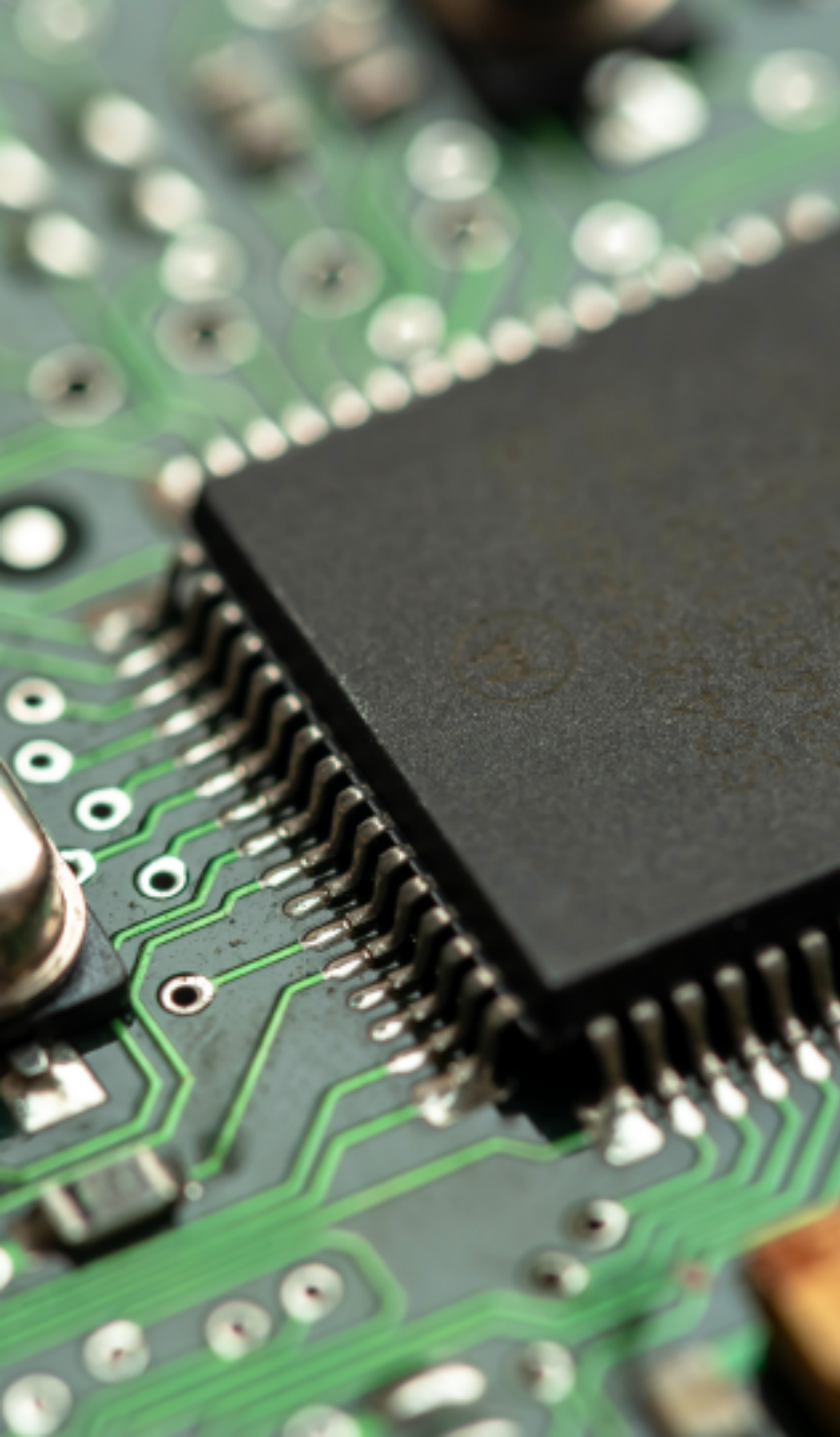




II2001 Mikroprozessortechnik

Lerneinheit 8: Kommunikationsschnittstellen

Ing. Jakob Czekansky, M.Sc.

Sommersemester 2026

Technische Hochschule Mittelhessen

Ingenieur-Informatik (B. Sc.)

Informatik (B. Sc.)

Inhalt

- I2C
 - Übertragungsgeschwindigkeit
 - Datenkodierung
 - Adressierung
- SPI
 - Verkabelung
- CAN
 - Verkabelung
 - Der CAN Frame
 - Arbitrierung
 - Bit Stuffing

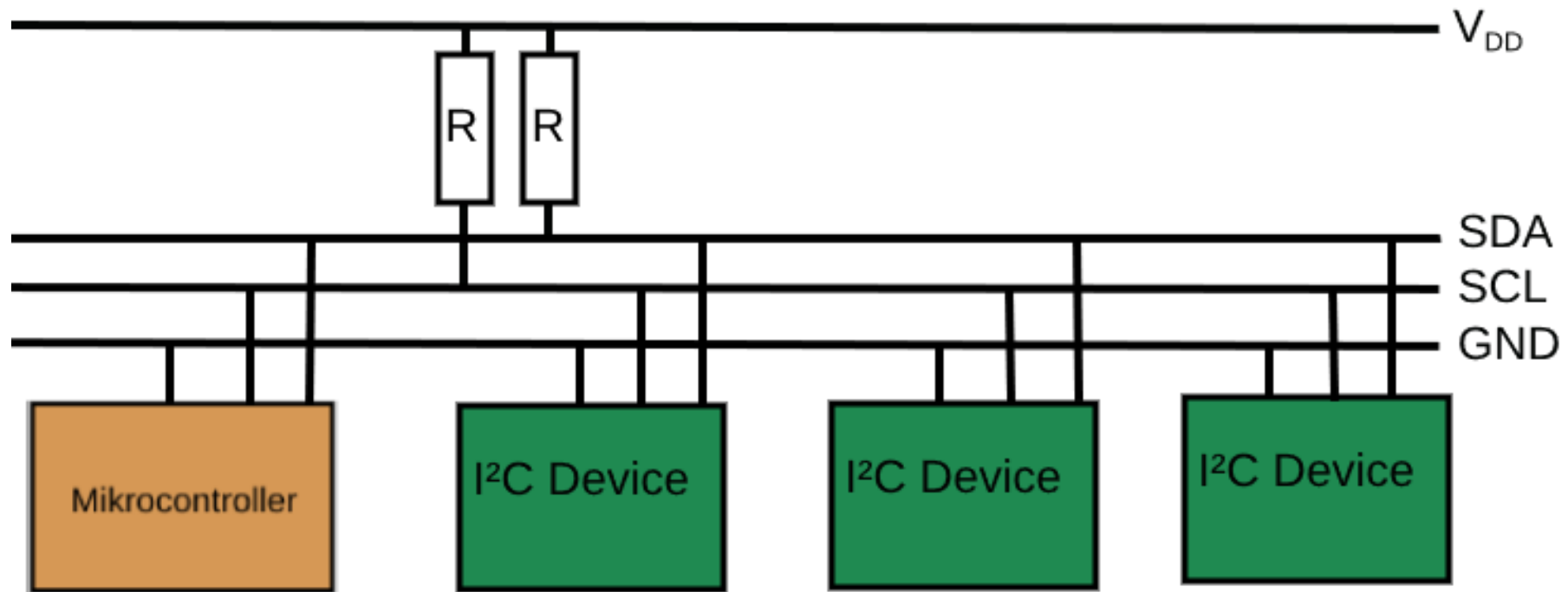
I²C (Inter-Integrated-Circuit)

Meist kurz als I²C oder I2C bezeichnet. Dieses Protokoll wurde von Fa. Philips entwickelt.

Übersetzt bedeutet der Name "zwischen (inter) integrierten Schaltungen (integrated circuits)".

I²C wird auch als **Two Wire Interface (TWI)** bezeichnet, da das Protokoll (abzgl. der V_{DD} und GND Leitungen) lediglich 2 Leitungen braucht.

Es gibt immer einen **Controller**, der den Bus leitet. Angeschlossen werden können allerdings mehrere sog. **Peripherien**.



Übertragungsgeschwindigkeit

Die Übertragungsgeschwindigkeit ist im Standardmodus mit USART vergleichbar, wobei I²C in anderen Modi mit bis zu 5MBit/s weit schneller sein kann.

Mode	Maximum speed	Direction
Standard mode (Sm)	100 kbit/s	Bidirectional
Fast mode (Fm)	400 kbit/s	Bidirectional
Fast mode plus (Fm+)	1 Mbit/s	Bidirectional
High-speed mode (Hs)	1.7 Mbit/s	Bidirectional
High-speed mode (Hs)	3.4 Mbit/s	Bidirectional
Ultra-fast mode (UFm)	5 Mbit/s	Unidirectional

Datenkodierung

Das I²C Protokoll verwendet eine Datenleitung (Serial Data, SDA), sowie eine durch den **Controller** (Host) vorgegebene Clock Leitung (Serial Clock, SCL).

- Es gibt jeweils ein Start (fallende Flanke) und ein Stop (steigende Flanke) Bit.
- Jedes Datenbyte des Senders wird mit einem ACK/NACK Bit angenommen/abgelehnt.
- Der Anfang einer Nachricht besteht aus zwei Teilen:
 - Die Adresse der Peripherie, die angesprochen werden soll (7 oder 10 Bit)
 - Read (1) oder Write (0); Wie man mit der Peripherie kommuniziert (1 Bit)
- Nachdem die Adresse bestätigt ist, werden tatsächliche Daten gesendet (je 8 bit).

start	address	R/ $\overline{W}$	ACK/NACK	D0	ACK/NACK	D1	ACK/NACK	stop
1	7	1	1	8	1	8	1	1

Adressierung

Da ein Bit durch $R/\overline{W}$ belegt ist, bleiben lediglich 7 Bit zur Adressierung übrig (seltene Erweiterung: 10 Bit).

Damit sind bis zu 128 Adressen möglich. Peripherien haben grundsätzlich festgelegte Adressen, die manchmal angepasst werden können (z. B. durch einen Schalter).

Tatsächlich verwendet werden können aber nur **112** dieser Adressen, da der restliche Adressraum vorbelegt ("reserved") ist.

Serial Peripheral Interface

- Von der Fa. Motorola (heute Freescale) entwickelt
- Kann über kurze Distanzen ($< 50\text{cm}$) verwendet werden
- Eigene Leitung je Übertragungsrichtungen
- Es gibt einen zentralen Kommunikationsleiter (Controller)
- Alle weiteren SPI-Kommunizierende (Peripherien) werden über ein Freigabesignal (Chip Select, CS) aktiviert
- Datenraten bis zu 50 Mbit/s, theoretisch unbegrenzt

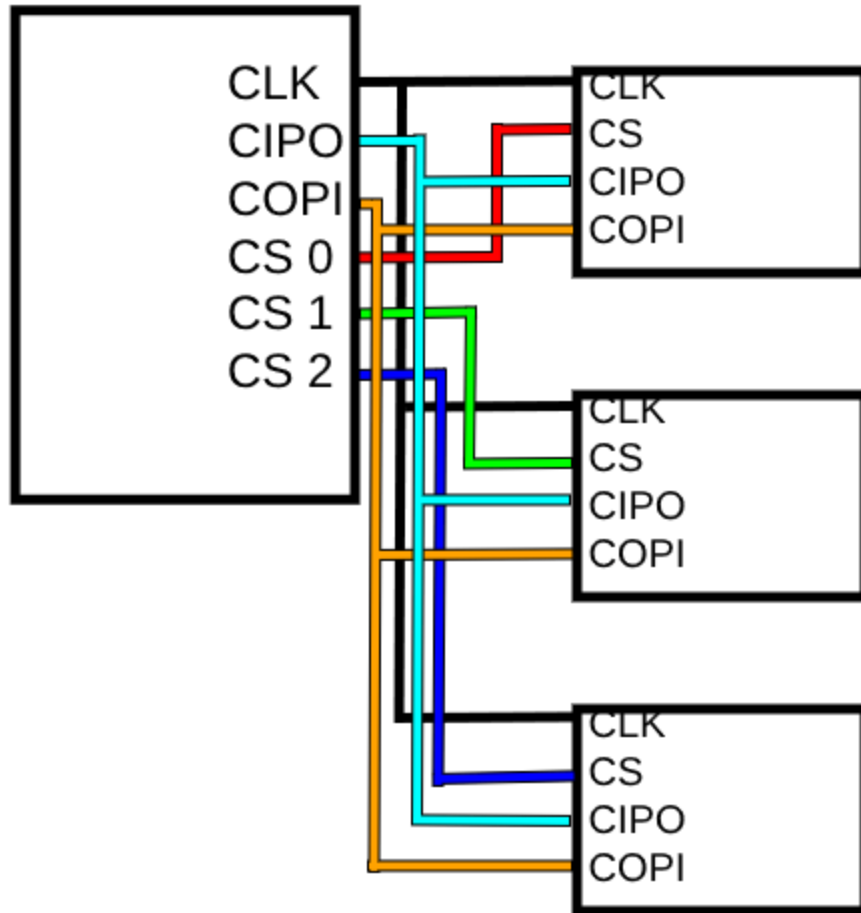
Verkabelung

Leitungen des SPI-Bus:

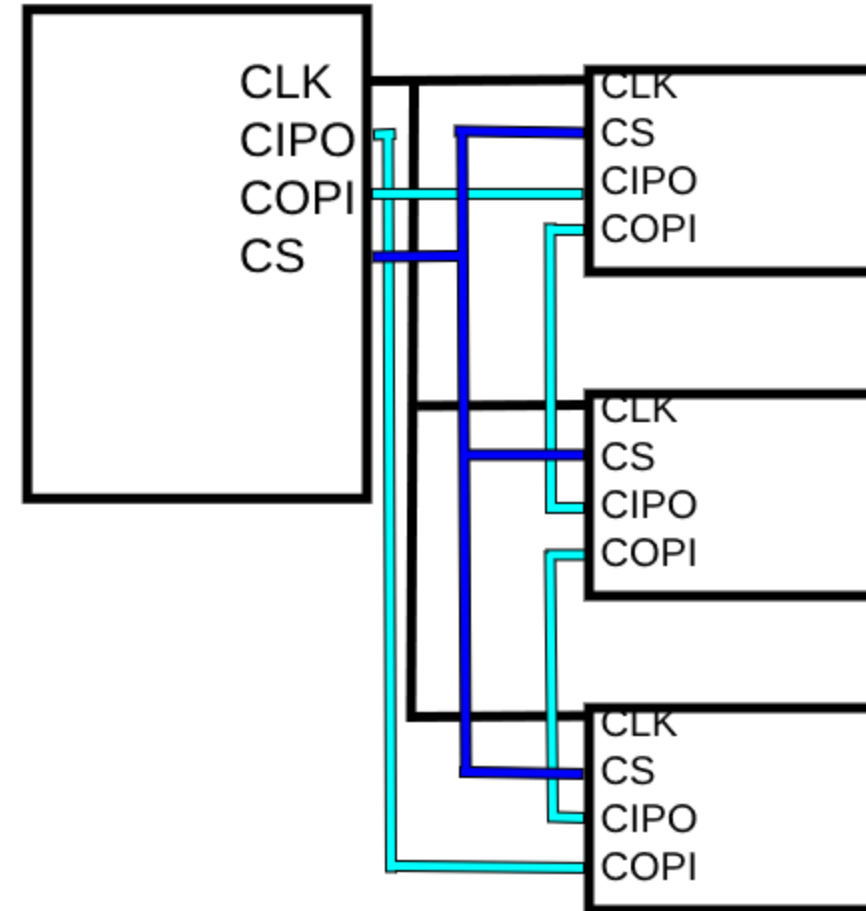
- **CIPO** (Controller In, Peripheral Out)
- **COPI** (Controller Out, Peripheral In)
- **SCK** (Serial Clock)
- **CS** (Chip Select)

Der SPI Bus lässt sich in zwei maßgeblichen Arten vernetzen.

- Im normalen Betrieb wird mit jeder Peripherie einzeln kommuniziert. Dementsprechend braucht jede Peripherie ein eigenes Chip Select. Diejenige Peripherie, für die das Chip Select Signal LOW gezogen ist, darf die CIPO und COPI Leitungen für sich beanspruchen.
- Beim sogenannten **Daisy Chaining**, geht jede Kommunikation durch alle Peripherien. Die Daten werden in dem Fall durchgereicht.



Normaler Betrieb



Daisy Chaining

CAN-Bus

Der CAN-Bus (Controller Area Network) ist ein von Bosch entwickeltes **dezentrales**, **nachrichtenbasiertes** Bussystem für die Automobilindustrie und findet wegen seiner Zuverlässigkeit auch in anderen Industrien weitreichende Verwendung.

- dezentral: Der CAN-Bus hat keinen Kommunikationskoordinator. Alle Teilnehmer sind "gleichberechtigt"
- nachrichtenbasiert: Nachrichten, statt Komponenten, werden mit Adressen versehen. Jeder Busteilnehmer entscheidet selbst, auf welche Nachrichtentypen er reagiert.

Verkabelung

Der Bus besteht lediglich aus den elektrisch Notwendigen (V_{DD} und GND) und zwei Leitungen für Datenübertragung (CANH, CANL).

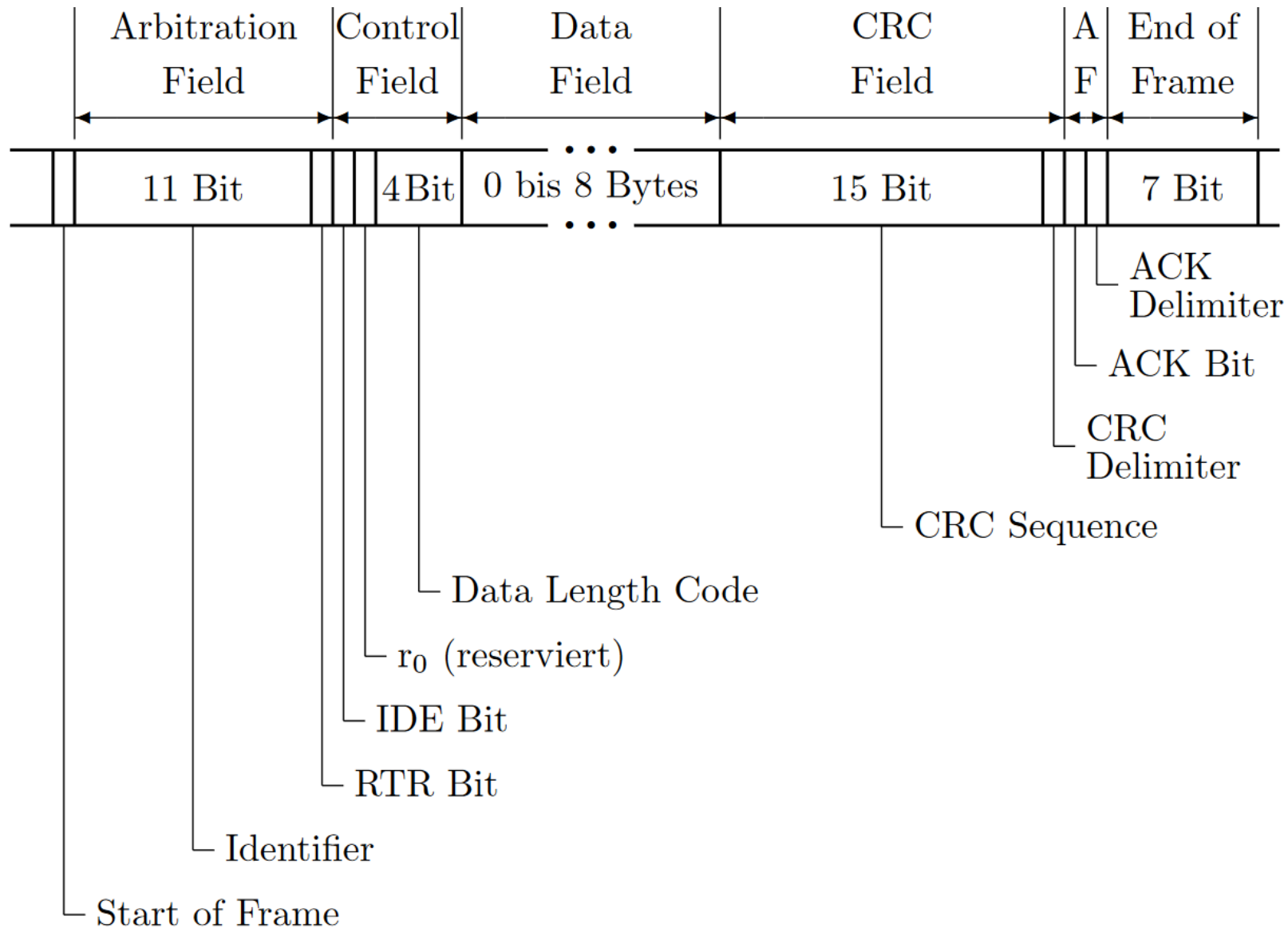
CAN ist durch die ineinander verdrehten Leitungen CANH und CANL sehr Fehlerresistent, da sie die sogenannte **symmetrische Datenübertragung** verwenden. (auch bei: HDMI, USB, Ethernet)



Externe Störungen wirken auf beide Leitungen identisch, wodurch diese aus den Datenleitungen, die das Inverse voneinander sind, herausgerechnet werden können.

Der CAN Frame

- **Nachrichten-ID:** wird zur Arbitrierung (Entscheidung wer darf senden), sowie zur Adressierung verwendet.
- **Kontrollfeld:** mit Information über die Übertragung, zB. wie lang ist das Datenfeld.
- **Daten:** bei bis zu 8 Bytes eine vergleichsweise große Payload, die dem Overhead gerecht wird.
- **CRC Prüfsumme:** zur Kontrolle der Richtigkeit der Daten.
- **ACK:** die Nachricht gilt als erfolgreich gesendet, wenn es mindestens einen erfolgreichen Empfänger gibt.
- **End Of Frame:** Durch 7 aufeinanderfolgende rezessive (0) Bits wird signalisiert, dass die Nachricht zuende ist und der Bus frei wird. Da rezessive Pegel verwendet werden, wirkt es funktional identisch dazu, den Busteilnehmer abzustecken.



Arbitrierung

Bei der Übersendung von Nachrichten kann es zu Kollisionen kommen, CAN löst diese auf, indem die Nachrichten-ID eine Priorität repräsentiert. (collision resolution)

- Eine niedrigere ID hat eine höhere Priorität und ein dominanter Pegel überwiegt immer gegenüber einem rezessiven Pegel auf dem Bus.
- Jeder Sender hört gleichzeitig mit, was auf dem Bus liegt.
- Wenn ein Sender etwas auf den Bus legt, das er nicht wieder auslesen kann, dann hört er auf zu senden und probiert es später erneut.
- So hören alle Sender auf zu senden, außer der mit der höchsten Priorität.

Bit Stuffing

Ein **End of Frame** (das Ende einer Nachricht) wird durch eine siebenfache Wiederholung des rezessiven Pegels identifiziert.

Wird innerhalb der Nachricht (z.B. im Datenfeld) dieses Muster verwendet, wird dieses als Nachrichtsende interpretiert.

Um das zu vermeiden wird nach fünf gleichen, aufeinanderfolgenden Bits ein sogenanntes **Stuff Bit** mit dem inversen Pegel in die Nachricht eingefügt. Da sich alle Teilnehmer auf dieses Verhalten einstellen, wird dieses Bit von allen ignoriert; die Endbedingung wird damit umgangen.